B

Academic Project Review 2

Design and Implementation of a Database Management System for Bank Account and Transaction Management System

Project 55 · Normalized Design, SQL Implementation & Database Prototype

Presented by
M.Srilakshmi Reddy

Course
AI-ML

Academic Year
2026

Review 2 — What Has Been Completed

14	3	2	7	54
Tables in 3NF	Triggers	Stored procedures	Reporting views	Ledger transactions

Review 1 → Review 2

- Review 1 ER diagram and initial schema converted to a **normalized 3NF design** with functional dependencies.
- Complete **DDL script** with PK, FK, UNIQUE, CHECK, ENUM and DEFAULT constraints.
- Business rules enforced inside the database by **triggers and stored procedures**.

Evidence in this review

- Sample data for 10 customers, 11 accounts, 8 transfers and 3 loans.
- Queries using **joins, subqueries, aggregation and views** — all executed, outputs shown.
- Rule-violation tests showing the database **rejecting** invalid operations.

Normalization Process — UNF to 3NF

UNF

One bank sheet: customer, branch, account, joint holders, many transactions and loan instalments repeated in a single record.

›

1NF

Atomic values; repeating groups (holders, transactions, instalments) moved to their own rows. Primary key added.

›

2NF

Partial dependencies removed. Branch and type data split from ACCOUNT; ACCOUNT_HOLDER resolves the M:N of joint accounts.

›

3NF

No transitive dependencies. Subtypes (TRANSFER, SERVICE_CHARGE, LOAN_REPAYMENT) hang off TRANSACTION; REPAYMENT allocates payments to schedule rows.

Example of a removed anomaly

- **Before:** account row stored branch name, city and manager — changing a manager meant editing every account.
- **After:** ACCOUNT stores only *branch_id*; BRANCH holds the details once.

Result

- No update, insert or delete anomalies.
- Every non-key attribute depends on **the key, the whole key, and nothing but the key**.

Functional Dependencies

Relation	Functional dependencies (determinant → dependents)
BRANCH	branch_id → branch_name, address, city, phone, branch_manager
ACCOUNT_TYPE	type_id → type_name, description, min_balance, is_interest_bearing
CUSTOMER	customer_id → full_name, date_of_birth, email, phone, address, id_proof
ACCOUNT	account_id → account_number, type_id, branch_id, status, open_date, balance; account_number → account_id
ACCOUNT_HOLDER	(account_id, customer_id) → holder_type, since_date
TRANSACTION	transaction_id → account_id, transaction_type, amount, transaction_date, description, balance_after
TRANSFER	transfer_id → debit_transaction_id, credit_transaction_id, remarks
SERVICE_CHARGE	charge_id → transaction_id, charge_type
BENEFICIARY	beneficiary_id → customer_id, beneficiary_name, account_number, bank_name, ifsc_code, type, created_at
STATEMENT	statement_id → account_id, from_date, to_date, generated_date
LOAN	loan_id → customer_id, account_id, loan_type, amount, interest_rate, start_date, end_date, status
LOAN_SCHEDULE	schedule_id → loan_id, due_date, due_amount, status; (loan_id, due_date) → schedule_id
LOAN_REPAYMENT	repayment_id → transaction_id, loan_id
REPAYMENT	repayment_id → schedule_id, transaction_id, amount, status

Every determinant is a candidate key (or superkey) of its relation, so each table satisfies BCNF as well as 3NF. Derived columns *account.balance* and *transaction.balance_after* are deliberate, trigger-maintained redundancy for fast statements.

Changes from Review 1 Schema to 3NF

Design decisions

- **TRANSFER** stores two references — debit and credit transaction — so every transfer is balanced by construction; amount is read from the ledger, not duplicated.
- **REPAYMENT** allocates a payment transaction to a LOAN_SCHEDULE instalment, so outstanding = scheduled − allocated.
- **Customer / branch / type details** kept in a single place and referenced by key.
- **ACCOUNT_HOLDER** resolves the many-to-many between customers and joint accounts.

Controlled redundancy

- *account.balance* and *transaction.balance_after* are computed by the BEFORE/AFTER INSERT triggers — never typed by users.
- Audit query confirms zero mismatches between the stored balance and the last ledger entry.
- Money columns use **DECIMAL(15,2)**, never floating point.

All 14 entities from the Review 1 ER diagram are retained — the ER diagram is unchanged.

Data Dictionary — Customers & Accounts

Table	Columns and constraints	Purpose
BRANCH	branch_id (PK), branch_name, address, city, phone, branch_manager	Bank branch details
ACCOUNT_TYPE	type_id (PK), type_name (UQ), min_balance ≥ 0, is_interest_bearing	Savings / Current / Salary / FD rules
CUSTOMER	customer_id (PK), full_name, date_of_birth, email (UQ), phone, id_proof (UQ)	Customer profile
ACCOUNT	account_id (PK), account_number (UQ), type_id FK, branch_id FK, status ENUM(ACTIVE, DORMANT, FROZEN, CLOSED), open_date, balance CHECK ≥ 0	Bank account
ACCOUNT_HOLDER	(account_id, customer_id) PK, holder_type ENUM(PRIMARY, JOINT), since_date	Joint-holder link
BENEFICIARY	beneficiary_id (PK), customer_id FK, beneficiary_name, account_number, bank_name, ifsc_code, type ENUM(INTERNAL, EXTERNAL)	Saved payees
STATEMENT	statement_id (PK), account_id FK, from_date, to_date, generated_date, CHECK to_date ≥ from_date	Statement log

Data Dictionary — Transactions & Loans

Table	Columns and constraints	Purpose
TRANSACTION	transaction_id (PK), account_id FK, transaction_type ENUM(7 types), amount CHECK > 0, transaction_date, description, balance_after	Single ledger of all money movement
TRANSFER	transfer_id (PK), debit_transaction_id FK (UQ), credit_transaction_id FK (UQ), remarks	Links the two legs of a transfer
SERVICE_CHARGE	charge_id (PK), transaction_id FK (UQ), charge_type ENUM(ATM_FEE, SMS_ALERT, ANNUAL_MAINTENANCE, …)	Fee details
LOAN	loan_id (PK), customer_id FK, account_id FK, loan_type, amount CHECK > 0, interest_rate, start_date, end_date, status	Loan master
LOAN_SCHEDULE	schedule_id (PK), loan_id FK, due_date, due_amount CHECK > 0, status ENUM(PENDING, PAID, OVERDUE), UQ(loan_id, due_date)	EMI schedule
LOAN_REPAYMENT	repayment_id (PK), transaction_id FK (UQ), loan_id FK	Repayment transaction subtype
REPAYMENT	repayment_id (PK), schedule_id FK, transaction_id FK, amount CHECK > 0, status	Allocates a payment to an instalment

Integrity Constraints — Business Rules Enforced

Business rule	How it is enforced in MySQL	Mechanism
Unique account / transaction IDs	PRIMARY KEY AUTO_INCREMENT on every ID; UNIQUE on account_number, email, id_proof	Constraint
Sufficient balance	trg_txn_before_insert rejects a debit if balance − amount < account type minimum balance	Trigger
Balanced debit / credit	sp_transfer posts both legs in one transaction; trg_transfer_before_insert checks equal amounts; v_transfers shows *balanced*	Procedure + trigger
Active-account requirement	trigger rejects any posting to DORMANT, FROZEN or CLOSED accounts	Trigger
Positive amounts	CHECK (amount > 0) on transaction, loan, schedule, repayment, plus trigger message	CHECK
Automatic balances	trg_txn_after_insert updates account.balance; balance_after stored per entry	Trigger
Atomic operations	sp_transfer and sp_loan_repayment use START TRANSACTION with EXIT HANDLER + ROLLBACK	Procedure

Transaction Posting Flow — Enforced in the Database

1. Request
INSERT into TRANSACTION, or CALL sp_transfer / sp_loan_repayment.

›

2. Validate
BEFORE INSERT trigger checks: amount > 0, account exists, status ACTIVE.

›

3. Balance check
Credit: new = balance + amount. Debit: new = balance − amount must be ≥ minimum balance.

›

4. Post
balance_after stored; AFTER trigger updates ACCOUNT.balance.

Transfer = one atomic unit

- Debit leg on the source account and credit leg on the destination are inserted inside one transaction.
- If either leg fails, **ROLLBACK** undoes both — money is never created or lost.

Loan repayment

- EMI is debited from the loan account and linked to the matching schedule row.
- When the instalment is fully paid its status becomes PAID automatically.

Sample Data & Implementation Progress

Table	Rows
branch	4
customer	10
account	11
holders	12
transaction	54
transfer	8
loan	3
schedule rows	27
repayments	8

Row counts read live from bank_db after running bank_review2.sql.

Implementation status

- **Done:** schema, constraints, indexes, triggers, procedures, sample data, views, queries, tests.
- Four branches, ten customers (one joint account), four account types, accounts in ACTIVE, FROZEN and CLOSED states.
- Loans: personal, vehicle and education, with EMI schedules generated by the standard formula.
- **Verified:** every account balance equals its last ledger balance.
- **Remaining (Review 3):** front-end application, final report, viva demo.

Queries — Joins & Subqueries

JOIN — account statement

```
SELECT t.transaction_id, t.transaction_date, t.transaction_type, t.amount,
t.balance_after
FROM `transaction` t JOIN account a ON a.account_id=t.account_id
WHERE a.account_number='50010000001001'
ORDER BY t.transaction_id LIMIT 6;
```

transaction_id	transaction_date	transaction_type	amount	balance_after
1	2026-07-01 10:15:00	DEPOSIT	50000.00	50000.00
11	2026-07-10 17:05:00	WITHDRAWAL	8000.00	42000.00
20	2026-08-18 10:10:00	DEPOSIT	22000.00	64000.00
24	2026-07-20 10:00:00	TRANSFER_DEBIT	5000.00	59000.00
31	2026-08-14 16:10:00	TRANSFER_CREDIT	6000.00	65000.00
37	2026-09-05 09:30:00	TRANSFER_CREDIT	10000.00	75000.00

SUBQUERY — above-average balance

```
SELECT account_number, balance FROM account
WHERE balance > (SELECT AVG(balance) FROM account WHERE status='ACTIVE')
ORDER BY balance DESC;
```

account_number	balance
50010000001003	500000.00
50010000003001	445438.88

CORRELATED SUBQUERY — customers without a loan

```
SELECT full_name FROM customer c
WHERE NOT EXISTS (SELECT 1 FROM loan l WHERE l.customer_id=c.customer_id)
LIMIT 4;
```

full_name
Divya Menon
Lakshmi Devi
Mohammed Imran
Priya Sharma

Queries — Aggregation & Branch Summary View

AGGREGATION — monthly deposits vs withdrawals

```
SELECT DATE_FORMAT(transaction_date,'%Y-%m') AS month,
 SUM(CASE WHEN transaction_type='DEPOSIT' THEN amount END) AS deposits,
 SUM(CASE WHEN transaction_type='WITHDRAWAL' THEN amount END) AS
withdrawals,
 COUNT(*) AS txns
FROM `transaction` GROUP BY month ORDER BY month;
```

month	deposits	withdrawals	txns
2026-06	—	—	1
2026-07	1140000.00	85000.00	22
2026-08	75000.00	44000.00	17
2026-09	146000.00	—	13
2026-10	—	—	1

VIEW — v_branch_summary

```
SELECT * FROM v_branch_summary;
```

branch_id	branch_name	city	accounts	total_deposits	avg_balance	inactive_accounts
1	Hanamkonda Main	Hanamkonda	4	683921.22	170980.31	1
2	Warangal Fort	Warangal	3	106941.00	35647.00	1
3	Hyderabad Banjara	Hyderabad	2	543438.88	271719.44	0
4	Karimnagar Town	Karimnagar	2	239624.28	119812.14	0

VIEW — v_account_balances (top 4)

account_number	type_name	branch_name	status	balance
50010000001003	Fixed Deposit	Hanamkonda Main	ACTIVE	500000.00
50010000003001	Current	Hyderabad Banjara	ACTIVE	445438.88
50010000004002	Savings	Karimnagar Town	ACTIVE	150624.28
50010000001001	Savings	Hanamkonda Main	ACTIVE	110157.22

Views — Transfers, Loan Outstanding & Repayments

v_transfers — each transfer is balanced (debited = credited)

transfer_id	transaction_date	from_account	to_account	debited	credited	balanced
1	2026-07-20 10:00:00	50010000001001	50010000001002	5000.00	5000.00	1
2	2026-07-25 14:30:00	50010000001002	50010000002001	7500.00	7500.00	1
3	2026-08-02 11:00:00	50010000003001	50010000002002	20000.00	20000.00	1
4	2026-08-14 16:10:00	50010000003002	50010000001001	6000.00	6000.00	1
5	2026-08-22 12:45:00	50010000002001	50010000004002	3000.00	3000.00	1

v_loan_outstanding — principal, EMI total, repaid, outstanding

loan_id	full_name	loan_type	principal	interest_rate	total_payable	total_repaid	outstanding	status
1	Ananya Reddy	PERSONAL	60000.00	12.00	61811.95	24724.78	37087.17	ACTIVE
2	Arjun Naidu	VEHICLE	300000.00	9.00	313668.08	114061.12	199606.96	ACTIVE
3	Karthik Goud	EDUCATION	150000.00	8.00	156066.46	28375.72	127690.74	ACTIVE

v_repayment_history (sample)

loan_id	full_name	due_date	due_amount	paid_on	paid_amount	instalment_status
1	Ananya Reddy	2026-08-01	12362.39	2026-08-01 10:00:00	12362.39	PAID
1	Ananya Reddy	2026-09-01	12362.39	2026-09-01 10:00:00	12362.39	PAID
2	Arjun Naidu	2026-07-01	28515.28	2026-07-01 09:00:00	28515.28	PAID
2	Arjun Naidu	2026-08-01	28515.28	2026-08-01 09:00:00	28515.28	PAID

Integrity Tests — Invalid Operations Are Rejected

```
INSERT INTO `transaction` (account_id, transaction_type, amount) VALUES
(1,'DEPOSIT',-500);
```
ERROR: Transaction amount must be positive

```
INSERT … (5,'WITHDRAWAL',9999999); -- exceeds balance
```
ERROR: Insufficient balance (minimum balance rule)

```
INSERT … (9,'DEPOSIT',1000); -- FROZEN account
```
ERROR: Account is not ACTIVE - transaction rejected

```
INSERT … (11,'DEPOSIT',1000); -- CLOSED account
```
ERROR: Account is not ACTIVE - transaction rejected

```
CALL sp_transfer(1, 1, 100, …); -- same account
```
ERROR: Cannot transfer to the same account

```
CALL sp_transfer(5, 2, 99999999, …); -- too large; both legs rolled back
```
ERROR: Insufficient balance (minimum balance rule)

```
INSERT INTO account (account_number, …) VALUES ('50010000001001', …);
```
ERROR: Duplicate entry '50010000001001' for key 'account_number'

Messages are the actual output of tests_review2.sql. Account balances were unchanged after every rejected statement.

Database Prototype Demonstration

Live demo flow

- Run **bank_review2.sql** — creates bank_db, tables, triggers, procedures, data and views.
- Show **v_account_balances** and an account statement with running balance.
- Call **sp_transfer** and show both ledger legs with matching amounts.
- Attempt an overdraw and a posting to a frozen account — both rejected.
- Show **v_loan_outstanding** before and after a repayment.

Next — Review 3

- Front-end application (Java / Python) connected to this database.
- Forms for customers, accounts, deposits, withdrawals, transfers and loans.
- Statements and branch reports using the existing views.
- Final report, test evidence and viva demonstration.

Files: bank_review2.sql · queries_review2.sql · tests_review2.sql

Thank You

Questions & Technical Discussion